

Lesson 1 ROS Introduction

1. An introduction to Robot Operating System (ROS)

Before learning ROS, the definition of the operating system needs to be understood first. What is the operating system? It provides some of software and hardware interfaces for direct use. Therefore, for different platforms and functions, different operating systems are needed to complete the underlying operating.

Ros is an open source operating system for robots. It provides OS-like services including hardware abstraction, low-level device control, implementation of commonly used functionality, message-passing between processes, and package management. Some tools and libraries are also provided for acquiring, compiling, writing, and executing programs for multi-machine fusion.

In addition, ROS has process-to-process communication, which is the biggest difference between it and other operating systems. In the traditional operating systems, communication between processes needs to be called by the system, which consumes a lot of system resources so that it is not suitable for robot operating system.

A robot may contain multiple sensors to collaborate through communication between modules to accomplish a given task through modules. Therefore, an effective and rapid communication between processes is essential for a robot, and ROS provides such function.



Distro	Release date	Poster	Tuturtle, turtle in tutorial	EOL date
ROS Noetic Ninjemys (Recommended)	May 23rd, 2020			May, 2025 (Focal EOL)
ROS Melodic Morenia	May 23rd, 2018			May, 2023 (Bionic EOL)
ROS Lunar Loggerhead	May 23rd, 2017			May, 2019
ROS Kinetic Kame	May 23rd, 2016			April, 2021 (Xenial EOL)
ROS Jade Turtle	May 23rd, 2015			May, 2017
ROS Indigo Igloo	July 22nd, 2014			April, 2019 (Trusty EOL)
ROS Hydro Medusa	September 4th, 2013			May, 2015

ROS Groovy Galapagos	December 31, 2012			July, 2014
ROS Fuerte Turtle	April 23, 2012			--
ROS Electric Emys	August 30, 2011			--
ROS Diamondback	March 2, 2011			--
ROS C Turtle	August 2, 2010			--
ROS Box Turtle	March 2, 2010			--

2. ROS Features

- 1) Node to node design and independ functions.
- 2) Simple stuture and high integration.
- 3) Pentiful tool packages and supporting multiple programming languages. Each of packages adopts different programming language.
- 4) Free and open source
- 5) Abundant communities

Official website: <https://wiki.ros.org>

Follow the instruction of this tutorial to setup software source
<https://wiki.ros.org/melodic/Installation/Ubuntu>

3.3 Update Software Package

Enter command “**sudo apt update**”.

```
ubuntu@ubuntu-virtual-machine:~$ sudo apt update
```

4. Install ROS

Open a command line terminal. Enter command “**sudo apt install ros-melodic-desktop-full**” to install ROS. The installation process generally needs to take around 10 min, depending on the network status.

```
ubuntu@ubuntu-virtual-machine:~$ sudo apt install ros-melodic-desktop-full
```

5. Set the Environment Variables

Open a command line terminal. Enter command “**echo "source /opt/ros/melodic/setup.bash" >> ~/.bashrc**” and “**echo "source /opt/ros/melodic/setup.bash" >> ~/.bashrc**” in sequence to complete the configuration for environment variables.

```
ubuntu@ubuntu-virtual-machine:~$ echo "source /opt/ros/melodic/setup.bash" >> ~/.bashrc
ubuntu@ubuntu-virtual-machine:~$ source ~/.bashrc
```

6. Install rosdep

Enter command “**sudo apt-get install python3-rosdep**” and press Enter to install rosdep. When the installation confirmation prompt occurs, enter “**Y**”.

```
ubuntu@ubuntu-virtual-machine:~$ sudo apt-get install python3-rosdep
[sudo] password for ubuntu:
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  python3-catkin-pkg-modules python3-docutils python3-pygments
  python3-pyparsing python3-roman python3-rosdep-modules
  python3-rosdistro-modules python3-rospkg-modules
Suggested packages:
  fonts-linuxlibertine | ttf-linux-libertine texlive-lang-french
  texlive-latex-base texlive-latex-recommended python-pyparsing-doc
The following packages will be REMOVED:
  python-rosdep
The following NEW packages will be installed:
  python3-catkin-pkg-modules python3-docutils python3-pygments
  python3-pyparsing python3-roman python3-rosdep python3-rosdep-modules
  python3-rosdistro-modules python3-rospkg-modules
0 upgraded, 9 newly installed, 1 to remove and 659 not upgraded.
Need to get 1,153 kB of archives.
After this operation, 6,290 kB of additional disk space will be used.
Do you want to continue? [Y/n] Y
```

7. Initialization

Enter command “**cd /etc/ros/rosdep/sources.list.d**” and press Enter to enter the “**cd /etc/ros/rosdep/sources.list.d**” folder.

```
ubuntu@ubuntu-virtual-machine:~$ cd /etc/ros/rosdep/sources.list.d
```

- 1) Enter command “**sudo gedit 20-default.list**” and press Enter to edit this file.

```
ubuntu@ubuntu-virtual-machine:/etc/ros/rosdep/sources.list.d$ sudo gedit 20-default.list
```

- 2) Replace the content in the original file with the below content. (Do not manually enter the content. Please directly copy and paste, and do not need)

```
# os-specific li# os-specific listings first yam
https://gitee.com/wybro/rostdistro/raw/master/rosdep/osx-homebrew.yaml osx # generic yam
https://gitee.com/wybro/rostdistro/raw/master/rosdep/base.yaml yam
https://gitee.com/wybro/rostdistro/raw/master/rosdep/python.yaml yam
https://gitee.com/wybro/rostdistro/raw/master/rosdep/ruby.yaml gbpdistro
```


<https://gitee.com/wybro/rostdistro/raw/master/releases/fuerte.yaml> fuerte # newer distributions (Groovy, Hydro, ...) must not be listed anymore, they are being fetched from the rostdistro index.yaml instead

During the modification, a warning will appear in terminal. It is a normal phenomena that can be ignored.

```

** (gedit:5086): WARNING **: 11:27:56.819: Set document metadata failed: Setting attribute metadata::gedit-position not supported
ubuntu@ubuntu-virtual-machine:/etc/ros/rosdep/sources.list.d$ sudo gedit 20-default.list

** (gedit:5129): WARNING **: 11:29:55.173: Set document metadata failed: Setting attribute metadata::gedit-spell-language not supported

** (gedit:5129): WARNING **: 11:29:55.174: Set document metadata failed: Setting attribute metadata::gedit-encoding not supported

** (gedit:5129): WARNING **: 11:29:57.356: Set document metadata failed: Setting attribute metadata::gedit-position not supported
ubuntu@ubuntu-virtual-machine:/etc/ros/rosdep/sources.list.d$

```

3) After the replacement is done, click “Save” to save the file, then close.



```

# os-specific listings first yml https://gitee.com/wybro/rostdistro/raw/master/rostdistro/osx-homebrew.yaml osx # generic yml https://gitee.com/wybro/rostdistro/raw/master/rostdistro/base.yaml yml https://gitee.com/wybro/rostdistro/raw/master/rostdistro/python.yaml yml https://gitee.com/wybro/rostdistro/raw/master/rostdistro/ruby.yaml gbpdistro https://gitee.com/wybro/rostdistro/raw/master/releases/fuerte.yaml fuerte # newer distributions (Groovy, Hydro, ...) must not be listed anymore, they are being fetched from the rostdistro index.yaml instead

```

4) Enter command “**cd /usr/lib/python2.7/dist-packages/rosdep2/**” and press Enter to get access to the “**rosdep2**” directory.

```

ubuntu@ubuntu-virtual-machine:/usr/lib/python2.7/dist-packages/rosdep2$ cd /usr/lib/python2.7/dist-packages/rosdep2/

```

5) Enter command “**sudo gedit gbpdistro_support.py**” and press Enter.

You might be prompted to enter the password of the virtual machine. Then press Enter to open the “**gbpdistro_support.py**” file and locate the “**FUERTE_GBPDISTRO_URL**” code.

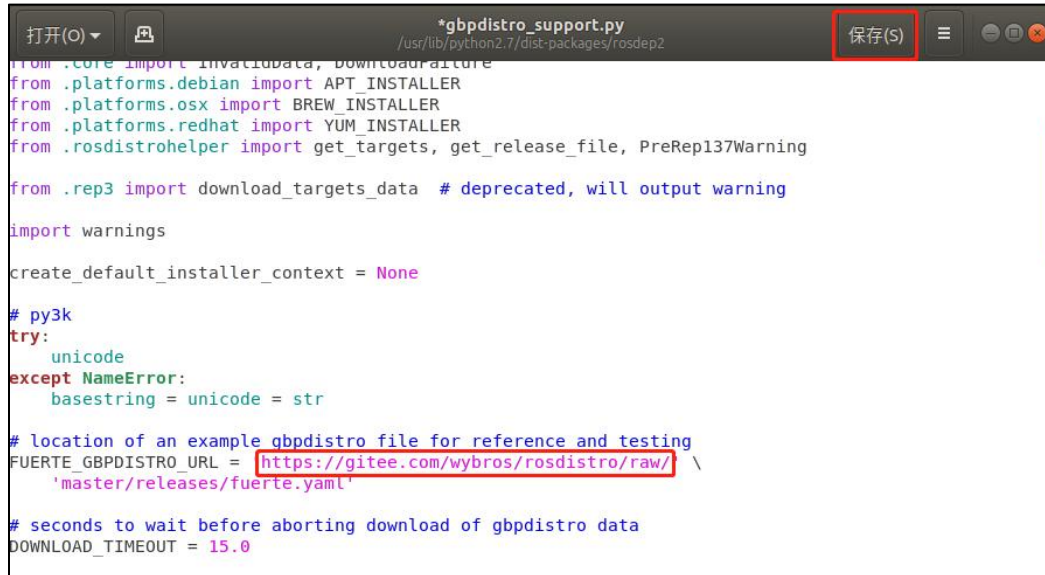
```

ubuntu@ubuntu-virtual-machine:/usr/lib/python2.7/dist-packages/rosdep2$ sudo gedit gbpdistro_support.py

```

```
# location of an example gbpdistro file for reference and testing
FUERTE_GBPDISTRO_URL = 'https://raw.githubusercontent.com/ros/rosdistro/' \
    'master/releases/fuerte.yaml'
```

- 6) Change “https://raw.githubusercontent.com/ros/rosdistro/” to “https://gitee.com/wybro/s/rosdistro/raw/”. (Only the content shown in following red box requires to modify) Then click “Save” and close the file.



```
*gbpdistro_support.py
/usr/lib/python2.7/dist-packages/rosdep2

from .core import InvalidData, DownloadFailure
from .platforms.debian import APT_INSTALLER
from .platforms.osx import BREW_INSTALLER
from .platforms.redhat import YUM_INSTALLER
from .rosdistrohelper import get_targets, get_release_file, PreRep137Warning

from .rep3 import download_targets_data # deprecated, will output warning

import warnings

create_default_installer_context = None

# py3k
try:
    unicode
except NameError:
    basestring = unicode = str

# location of an example gbpdistro file for reference and testing
FUERTE_GBPDISTRO_URL = 'https://gitee.com/wybro/s/rosdistro/raw/' \
    'master/releases/fuerte.yaml'

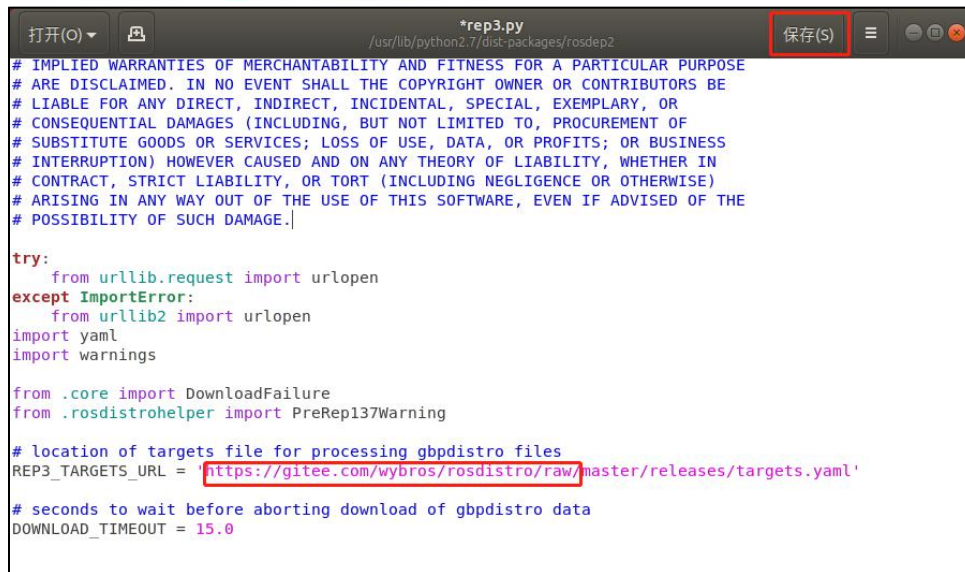
# seconds to wait before aborting download of gbpdistro data
DOWNLOAD_TIMEOUT = 15.0
```

- 7) Enter command “**sudo gedit rep3.py**” and press Enter to open the “rep3.py” file and locate the “REP3_TARGETS_URL” code.

```
ubuntu@ubuntu-virtual-machine:/usr/lib/python2.7/dist-packages/rosdep2$ sudo gedit rep3.py
```

```
# location of targets file for processing gbpdistro files
REP3_TARGETS_URL = 'https://raw.githubusercontent.com/ros/rosdistro/master/releases/targets.yaml'
```

- 8) Change “https://raw.githubusercontent.com/ros/rosdistro/” to “https://gitee.com/wybro/s/rosdistro/raw/” (Only the content shown in following red box requires to modify) Then click “Save” and close the file.



```

打开(O)  /usr/lib/python2.7/dist-packages/rosdep2  保存(S)
# IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE
# ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE
# LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR
# CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF
# SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS
# INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN
# CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE)
# ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE
# POSSIBILITY OF SUCH DAMAGE.

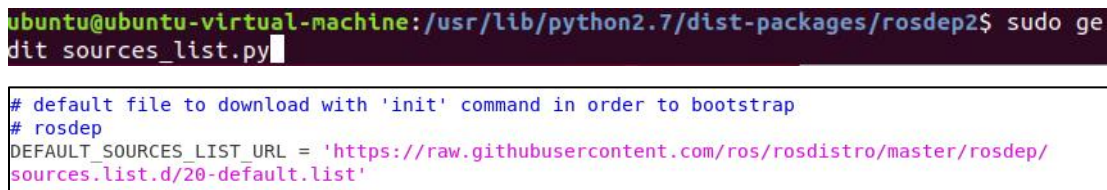
try:
    from urllib.request import urlopen
except ImportError:
    from urllib2 import urlopen
import yaml
import warnings

from .core import DownloadFailure
from .rostdistrohelper import PreRep137Warning

# location of targets file for processing gbpdistro files
REP3_TARGETS_URL = 'https://gitee.com/wybro/s/rostdistro/raw/master/releases/targets.yaml'

# seconds to wait before aborting download of gbpdistro data
DOWNLOAD_TIMEOUT = 15.0
    
```

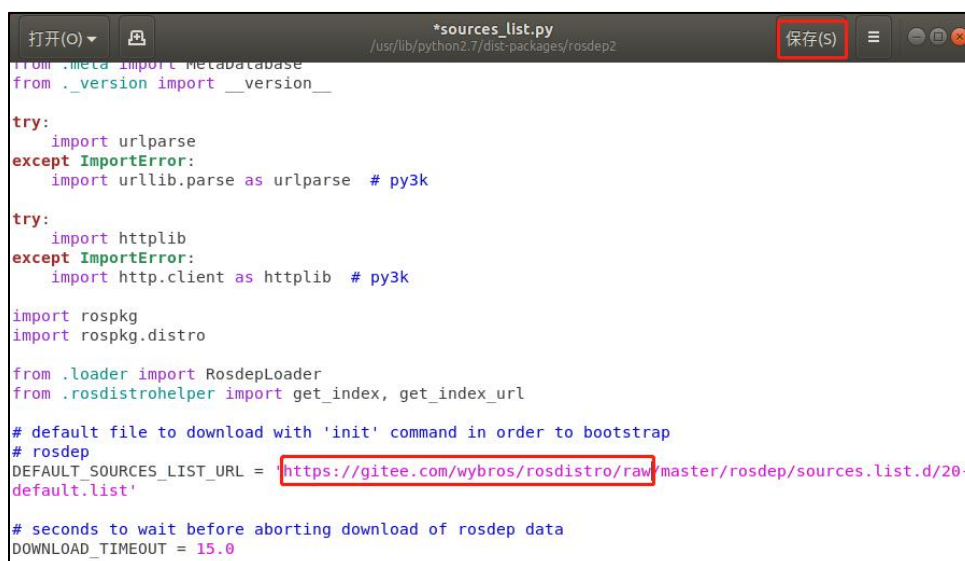
- 9) Enter command “**sudo gedit sources_list.py**” and press Enter to open the “**sources_list.py**” file and locate the “**DEFAULT_SOURCES_LIST_URL**” code.



```

ubuntu@ubuntu-virtual-machine: /usr/lib/python2.7/dist-packages/rosdep2$ sudo ge
dit sources_list.py
# default file to download with 'init' command in order to bootstrap
# rosdep
DEFAULT_SOURCES_LIST_URL = 'https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/
sources.list.d/20-default.list'
    
```

- 10) Change “**https://raw.githubusercontent.com/ros/rosdistro/**” to “**https://gitee.com/wybro/s/rostdistro/raw/**”. (Only the content shown in following red box requires to modify) Then click “Save” and close the file.



```

打开(O)  /usr/lib/python2.7/dist-packages/rosdep2  保存(S)
from .meta import Metadatabase
from .version import __version__

try:
    import urlparse
except ImportError:
    import urllib.parse as urlparse # py3k

try:
    import httplib
except ImportError:
    import http.client as httplib # py3k

import rospkg
import rospkg.distro

from .loader import RosdepLoader
from .rostdistrohelper import get_index, get_index_url

# default file to download with 'init' command in order to bootstrap
# rosdep
DEFAULT_SOURCES_LIST_URL = 'https://gitee.com/wybro/s/rostdistro/raw/master/rosdep/sources.list.d/20-
default.list'

# seconds to wait before aborting download of rosdep data
DOWNLOAD_TIMEOUT = 15.0
    
```


- 11) Enter command “**cd /usr/lib/python2.7/dist-packages/rosdistro/**” and press Enter to get access to the “**rosdistro**” directory.

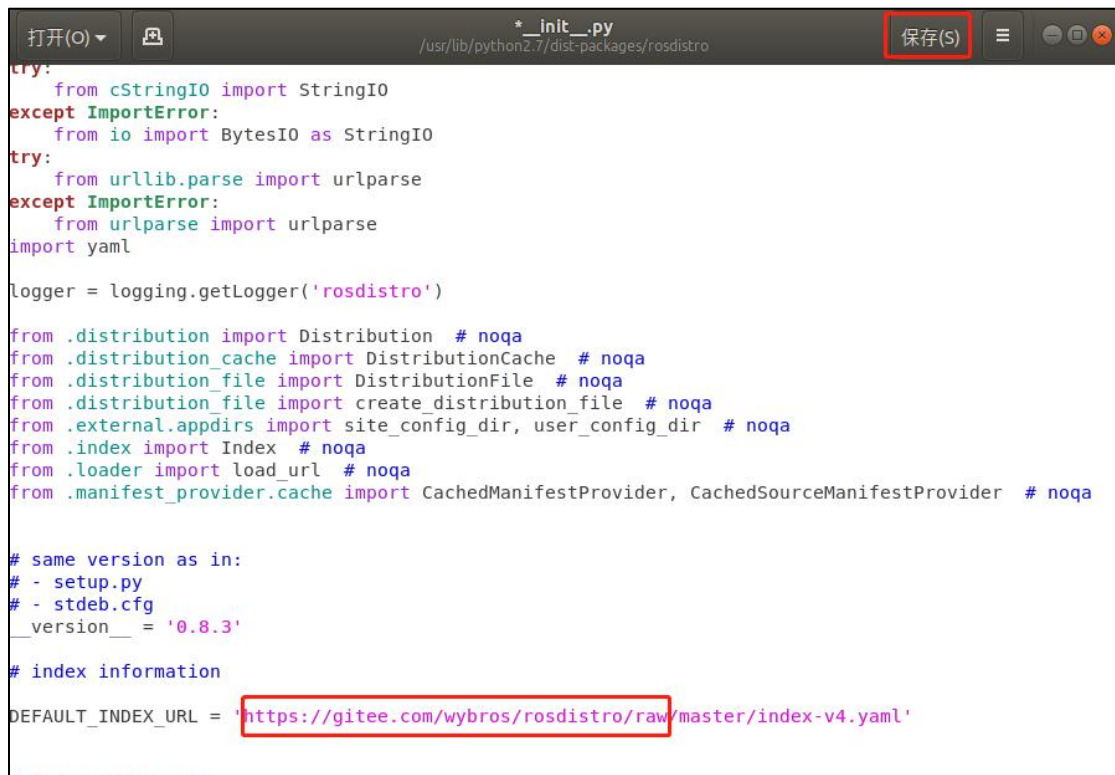
```
ubuntu@ubuntu-virtual-machine:/usr/lib/python2.7/dist-packages/rosdep2$ cd /usr/lib/python2.7/dist-packages/rosdistro/
```

- 12) Enter command “**sudo gedit __init__.py**” and press Enter to get access to “**__init__.py**” file, and locate the “**DEFAULT_INDEX_URL**” code.

```
ubuntu@ubuntu-virtual-machine:/usr/lib/python2.7/dist-packages/rosdistro$ sudo gedit __init__.py
```

```
# index information
DEFAULT_INDEX_URL = 'https://raw.githubusercontent.com/ros/rosdistro/master/index-v4.yaml'
```

- 13) Change “**https://raw.githubusercontent.com/ros/rosdistro/**” to “**https://gitee.com/wybro/s/rosdistro/raw/**”. (Only the content shown in following red box requires to modify) Then click “Save” and close the file.



```
try:
    from cStringIO import StringIO
except ImportError:
    from io import BytesIO as StringIO
try:
    from urllib.parse import urlparse
except ImportError:
    from urlparse import urlparse
import yaml

logger = logging.getLogger('rosdistro')

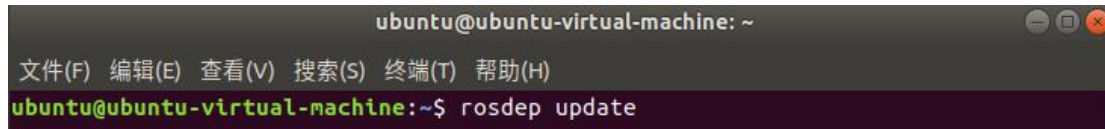
from .distribution import Distribution # noqa
from .distribution_cache import DistributionCache # noqa
from .distribution_file import DistributionFile # noqa
from .distribution_file import create_distribution_file # noqa
from .external.appdirs import site_config_dir, user_config_dir # noqa
from .index import Index # noqa
from .loader import load_url # noqa
from .manifest_provider.cache import CachedManifestProvider, CachedSourceManifestProvider # noqa

# same version as in:
# - setup.py
# - stdeb.cfg
__version__ = '0.8.3'

# index information
DEFAULT_INDEX_URL = 'https://gitee.com/wybro/s/rosdistro/raw/master/index-v4.yaml'
```

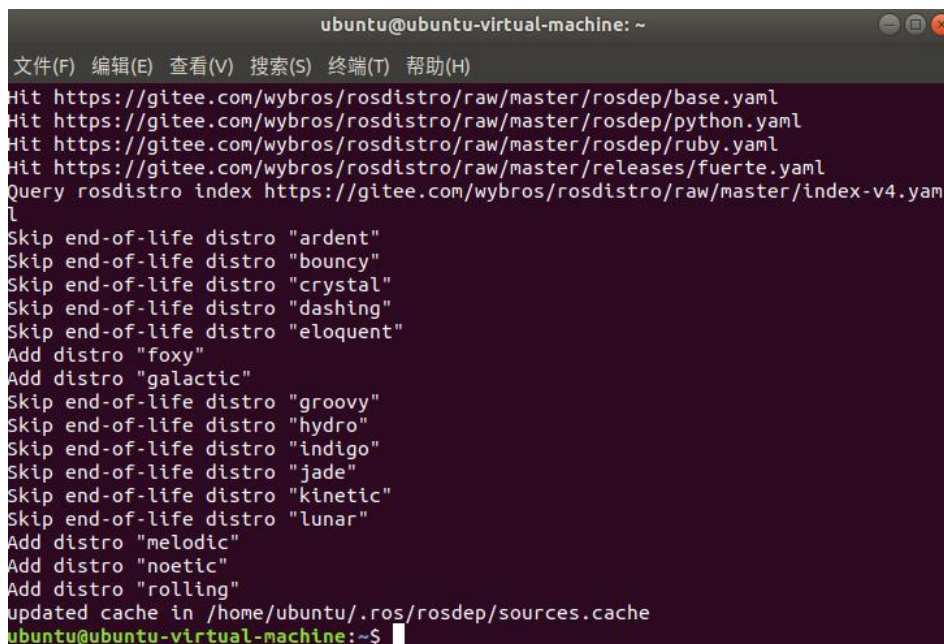
8. Update

Open a new command line terminal and enter command "**rosdep update**".



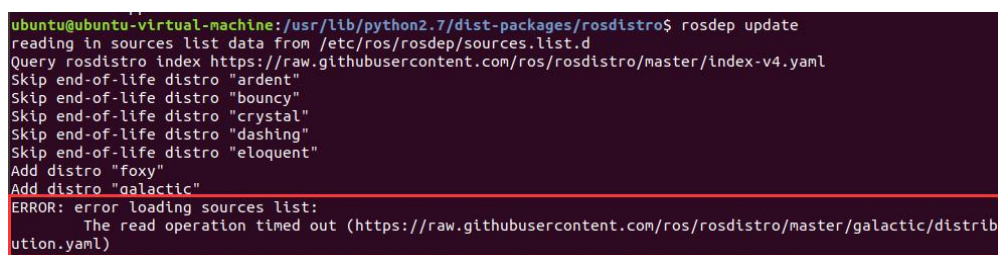
```
ubuntu@ubuntu-virtual-machine: ~  
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)  
ubuntu@ubuntu-virtual-machine:~$ rosdep update
```

The below information represents the update is successfully complete.



```
ubuntu@ubuntu-virtual-machine: ~  
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)  
Hit https://gitee.com/wybro/rostdistro/raw/master/rosdep/base.yaml  
Hit https://gitee.com/wybro/rostdistro/raw/master/rosdep/python.yaml  
Hit https://gitee.com/wybro/rostdistro/raw/master/rosdep/ruby.yaml  
Hit https://gitee.com/wybro/rostdistro/raw/master/releases/fuerte.yaml  
Query rostdistro index https://gitee.com/wybro/rostdistro/raw/master/index-v4.yaml  
Skip end-of-life distro "ardent"  
Skip end-of-life distro "bouncy"  
Skip end-of-life distro "crystal"  
Skip end-of-life distro "dashing"  
Skip end-of-life distro "eloquent"  
Add distro "foxy"  
Add distro "galactic"  
Skip end-of-life distro "groovy"  
Skip end-of-life distro "hydro"  
Skip end-of-life distro "indigo"  
Skip end-of-life distro "jade"  
Skip end-of-life distro "kinetic"  
Skip end-of-life distro "lunar"  
Add distro "melodic"  
Add distro "noetic"  
Add distro "rolling"  
updated cache in /home/ubuntu/.ros/rosdep/sources.cache  
ubuntu@ubuntu-virtual-machine:~$
```

If you encounter a timeout error as shown in the figure, it may be due to network issues. In this case, you can try entering "**rosdep update**" multiple times to attempt updating. If it still fails to update, you should check the contents of "**1.7 Initialization**" to ensure that the modifications made are correct. After verifying the changes, the user can try running "**rosdep update**" again to perform the update.



```
ubuntu@ubuntu-virtual-machine:/usr/lib/python2.7/dist-packages/rostdistro$ rosdep update  
reading in sources list data from /etc/ros/rosdep/sources.list.d  
Query rostdistro index https://raw.githubusercontent.com/ros/rosdistro/master/index-v4.yaml  
Skip end-of-life distro "ardent"  
Skip end-of-life distro "bouncy"  
Skip end-of-life distro "crystal"  
Skip end-of-life distro "dashing"  
Skip end-of-life distro "eloquent"  
Add distro "foxy"  
Add distro "galactic"  
ERROR: error loading sources list:  
The read operation timed out (https://raw.githubusercontent.com/ros/rosdistro/master/galactic/distribution.yaml)
```

Lesson 3 ROS Documents and Terms

Instruction

1. The Composition of ROS File System

ROS file consists of Packages and Manifests (package.xml).

Packages: The function packages, the most basic unit of the Ros software, contains node source code, configuration files, data definition, etc.

Manifest (package xml): The description file of function package is used to define dependencies between the meta information related to the function package, including version, maintainer, license agreement, etc.

2. Understand the Basic Terms of ROS

The following table provides explanation on some of ROS basic terms.

Terms	Instruction
ROS Master	The connection and message communication between nodes.
Node	The smallest processing unit running in Ros is usually an executable file. Each node can use topic or service to communicate with other nodes.
Message	The variables of int, float, boolean and other data types.
Topic	A one-way asynchronous communication mechanism. The data can be transmitted between nodes through publishing message to topics or subscribing topics. The

	type of topics are determined by that of corresponding message.
Publish	Send data with the message type corresponding to the topic content.
Publisher	To execute publishing, the publisher nodes register their topics and other various information on the master node, and send messages to subscriber nodes that wish to subscribe.
Subscrib	Receive data with the message type corresponding to the topic content.
Subscriber	To execute subscribing, the subscriber node registers its own topic and other various informations on the master node and receives the inforamtion from the master node about all publisher nodes that have published the topic to which this node needs to subscribe.
Service	A two-way synchronous communication mechanism. The service client requests the service corresponding to the specific purpose task and the service server replys to the service.
Service Server	The node with request as input and respond as output
Service Client	The node with respond as input and request as output

3. Learn about ROS Common Files

Terms	Instruction
urdf file	It is a model file describing all elements of robot including link, joint, axis, dynamics, visual and collision.
Srv file	It is stored in srv folder for ROS service message definition and contains two parts, request and reply. The symbol "--" is used to separate the request from the reply.
Msg file	It is stored in msg folder and used for ROS topics message definitions.
package.xml	Describe the property of function package including its name, version, author, etc.
CmakeLists.txt	Compile configuration files by using Cmake.
Launch file	Launch file. It contains the node and services needed to systematically boot up the robot.

4. Learn About Common Commands

ROS commands can be divided into five types: ROS shell commands, ROS execution commands, ROS information commands, ROS catkin commands, ROS function package commands. The followings are some of commonly used basic commands:

◆ ROS shell commands

Command	Explanation	Instruction
roscd	ros+cd(changes directory)	Come to the specified ROS package directory.
rosls	ros+ls(lists files)	Display the file and directory of ROS package
roscd	ros+ed(editor)	Edit the files of ROS package
roscp	ros+cp(copies files)	Copy the files of ROS package
rospd	ros+pushd	Add directory to ROS directory index
roscd	ros+directory	Display the directory in ROS directory index.

◆ ROS Executing Command

Command	Explanation	Instruction
roscore	ros+core	Boot ROS master node management
roslaunch	ros+run	Run ROS node
roslaunch	ros+launch	Start launch file
rosclean	rosclean	Check or delete ROS log file

◆ ROS Information command

Command	Explanation	Instruction
rostopic	ros+topic	Confirm ROS topics information
rosservice	ros+service	Confirm ROS service information
roscall	ros+node	Confirm Ros node information
rosparam	ros+param(parameter)	Confirm and modify ROS parameter information
rosviz	ros+bag	Record and replay ROS message
rosmmsg	ros+msg	Display ROS message type
rossrv	ros+srv	Display ROS service type
rosversion	ros+version	Display ROS package and version information
roswtf	ros+wtf	Check ROS system

◆ ROS catkin Command

Command	Instruction
catkin_create_pkg	Automatically generate function package

catkin_make	Build all function packages in diectory on the basis of catkin build system.
catkin_eclipse	Modify the package generated with the catkin build system to work in Eclipse environment.
catkin_prepare_release	Update CHANGELOG.rst file generated by “catkin_generate_changelog”
catkin_generate_changelog	Generate or upadate CHANGELOG.rst file when publishing
catkin_init_workspace	Initialize the workspace directory of the catkin build system
catkin_find	Find all warkspace directories in use

◆ **ROS Package Command**

Command	Explanation	Instruction
rospack	ros+pack(age)	View information related to ROS funcation package
roinstall	ros+install	Install the ROS add-on package
rosdep	ros+dep(endencies)	Install the dependency files for the specified function package

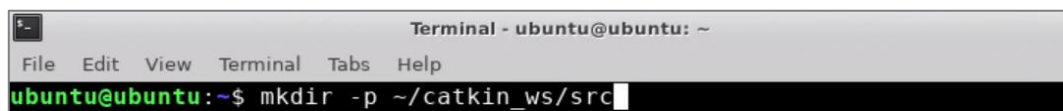
rosllocate	ros+locate	Display the information related to ROS function package
roscrcate-pkg	ros+create-pkg	Automatically generate ROS function package (Use for old rosbuidl system)
rosmake	ros+make	Build ROS function package (used for old rosbuidl system)

Lesson 4 Create Workspace and Package

The creation of messages and services, the writing of publishers and subscribers, the writing of servers and clients all operate on the basis of ROS function packages, so it is necessary to create workspace and function packages before performing the operations above.

1. Create Workspace

- 1) Open the terminal.
- 2) Input “mkdir -p ~/catkin_ws/src” command and press “Enter” to create a workspace named catkin_ws under the specified directory.



```
Terminal - ubuntu@ubuntu: ~  
File Edit View Terminal Tabs Help  
ubuntu@ubuntu:~$ mkdir -p ~/catkin_ws/src
```



2. Create ROS Package

- 1) Input “cd ~/catkin_ws/src” command, and then press “Enter” to come to the source file space (src) directory of catkin_ws.



```
Terminal - ubuntu@ubuntu: ~  
File Edit View Terminal Tabs Help  
ubuntu@ubuntu:~$ cd ~/catkin_ws/src
```

- 2) Input “catkin_create_pkg beginner_hiwonder std_msgs rospy roscpp” command, and then press “Enter” to create function package named

“beginner_hiwonder” and add dependencies (msgs, rospy, roscpp).

```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws/src$ catkin_create_pkg beginner_hiwonder std_msgs rospy roscpp
Created file beginner_hiwonder/package.xml
Created file beginner_hiwonder/CMakeLists.txt
Created folder beginner_hiwonder/include/beginner_hiwonder
Created folder beginner_hiwonder/src
Successfully created files in /home/ubuntu/catkin_ws/src/beginner_hiwonder. Please adjust the values
in package.xml.
```

3) Input “cd ~/catkin_ws” command and press “Enter” to back to workspace.

```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder$ cd ~/catkin_ws
```

4) Input “catkin_make” command and press “Enter” to complete the creation of function package in catkin workspace.

```
Terminal - ubuntu@ubuntu: ~/catkin_ws
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws$ catkin_make

-- traversing 1 packages in topological order:
-- ~ - beginner_hiwonder
--
-- +++ processing catkin package: 'beginner_hiwonder'
-- ==> add_subdirectory(beginner_hiwonder)
-- Configuring done
-- Generating done
-- Build files have been written to: /home/ubuntu/catkin_ws/build
####
#### Running command: "make -j4 -l4" in "/home/ubuntu/catkin_ws/build"
####
```

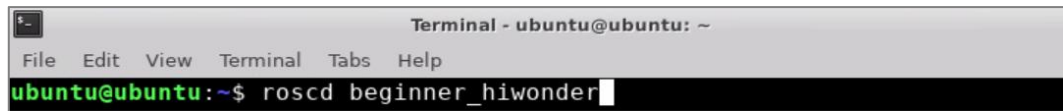
5) Input “echo “source ~/catkin_ws/devel/setup.bash” >> ~/.bashrc” command to add catkin_ws workspace to ROS enviroment, which the creation of function package is completed.

```
Terminal - ubuntu@ubuntu: ~
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~$ echo "source ~/catkin_ws/devel/setup.bash" >> ~/.bashrc
```

6) Input “source ~/.bashrc” command and press “Enter” to add the path.

```
Terminal - ubuntu@ubuntu: ~
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~$ source ~/.bashrc
```

- 7) Input “roscd beginner_hiwonder” command to enter the directory of funtion package and press “Enter” to verify whether the function package is created.

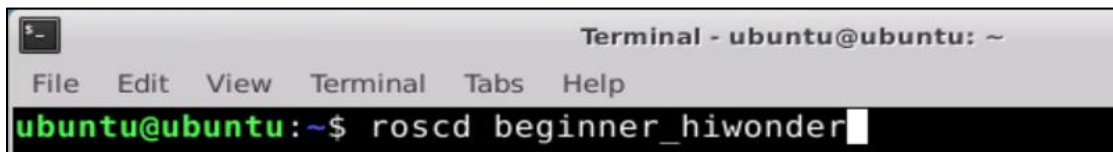


```
Terminal - ubuntu@ubuntu: ~
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~$ roscd beginner_hiwonder
```

Lesson 5 Write A Simple Publisher

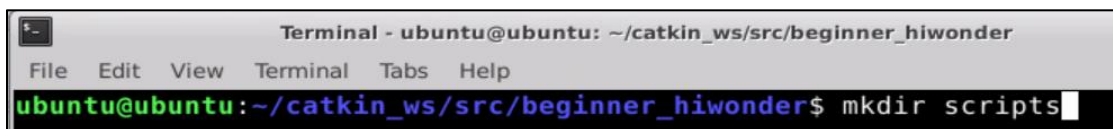
This section takes the creation of a velocity_publisher.py publisher node as an example to explain.

- 1) Input “roscd beginner_hiwonder” command and press “Enter” to enter beginner_hiwonder software package.



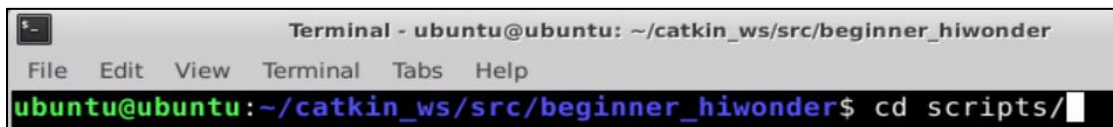
```
Terminal - ubuntu@ubuntu: ~  
File Edit View Terminal Tabs Help  
ubuntu@ubuntu:~$ roscd beginner_hiwonder
```

- 2) Input “mkdir scripts” command and press “Enter” to create a new scripts directory to store Python scripts.



```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder  
File Edit View Terminal Tabs Help  
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder$ mkdir scripts
```

- 3) Input “cd scripts/” command and press “Enter”.



```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder  
File Edit View Terminal Tabs Help  
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder$ cd scripts/
```

- 4) Input “vi velocity_publisher.py” command to edit program, and then copy the following program. If need to modify, you can press “i” again. After modifying, press “Esc” and input “:wq” to save and exit the file.



```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/scripts  
File Edit View Terminal Tabs Help  
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder/scripts$ vi velocity_publisher.py
```

```
#!/usr/bin/env python
```

```
# -*- coding: utf-8 -*-
```

```
#Publish this routine to the topic turtle1/cmd_vel, the type of message geometry_msgs::Twist
```

```
import rospy

from geometry_msgs.msg import Twist

def velocity_publisher():

    # Initialize ROS node

    rospy.init_node('velocity_publisher', anonymous=True)

    # Create a Publisher and publish a topic named /turtle1/cmd_vel. The type of message is
    geometry_msgs::Twist and the queue size is 10.

    turtle_vel_pub = rospy.Publisher('/turtle1/cmd_vel', Twist, queue_size=10)

    #set the loop rate

    rate = rospy.Rate(10)

    while not rospy.is_shutdown():

        # Initialize the message of geometry_msgs::Twist type

        vel_msg = Twist()

        vel_msg.linear.x = 0.5

        vel_msg.angular.z = 0.2

        # Publish message
```

```

turtle_vel_pub.publish(vel_msg)

rospy.loginfo("Publsh turtle velocity command[%0.2f m/s, %0.2f rad/s]",

              vel_msg.linear.x, vel_msg.angular.z)


# Delay on the basis of loop rate

rate.sleep()


if __name__ == '__main__':

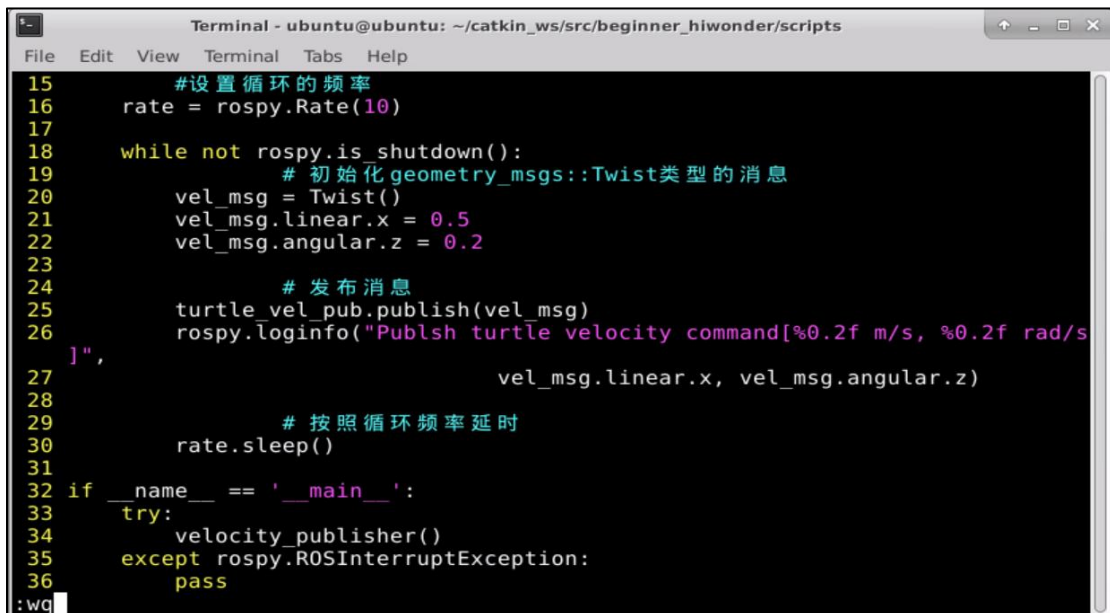
    try:

        velocity_publisher()

    except rospy.ROSInterruptException:

        pass

```



```

15 #设置循环的频率
16 rate = rospy.Rate(10)
17
18 while not rospy.is_shutdown():
19     # 初始化 geometry_msgs::Twist类型的消息
20     vel_msg = Twist()
21     vel_msg.linear.x = 0.5
22     vel_msg.angular.z = 0.2
23
24     # 发布消息
25     turtle_vel_pub.publish(vel_msg)
26     rospy.loginfo("Publsh turtle velocity command[%0.2f m/s, %0.2f rad/s",
27                   vel_msg.linear.x, vel_msg.angular.z)
28
29     # 按照循环频率延时
30     rate.sleep()
31
32 if __name__ == '__main__':
33     try:
34         velocity_publisher()
35     except rospy.ROSInterruptException:
36         pass
:wq

```

- 5) Input command “**chmod +x velocity_publisher.py**” to give executable permissions to the saved velocity_publisher.py.

A screenshot of a terminal window titled "Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/scripts". The terminal shows the command "chmod +x velocity_publisher.py" being entered at the prompt "ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder/scripts\$". The command is partially visible as "chmo" and "velocity_pu" on the first line, and "sher.py" on the second line.

```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/scripts
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder/scripts$ chmo
velocity_pu
sher.py
```

6) The publisher editing is complete.

Lesson 6 Write A Simple Subscriber

The creation of subscriber is based on the edited publisher. The subscription are only possible as long as the message is published. If you have not edited the publisher, you can view the content in “Lesson 4 Write A Simple Subscriber” and follow the steps to edit it.

1. Write Subscriber Node

This section takes the creation of a pose_subscriber.py node as an example to explain.

- 1) Input “cd catkin_ws/src/beginner_hiwonder/scripts/” command and press “Enter”.



```
Terminal - ubuntu@ubuntu: ~  
File Edit View Terminal Tabs Help  
ubuntu@ubuntu:~$ cd catkin_ws/src/beginner_hiwonder/scripts/
```

- 2) Enter “vi pose_subscriber.py” command to edit the program, and then copy the following program. If need to modify, press “i”. After modifying, press “Esc” and input “:wq” to save and exit.



```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/scripts  
File Edit View Terminal Tabs Help  
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder/scripts$ vi pose_subscriber.py
```

```
#!/usr/bin/env python  
  
# -*- coding: utf-8 -*-  
  
import rospy  
  
from turtlesim.msg import Pose
```

```
def poseCallback(msg):

    rospy.loginfo("Turtle pose: x:%0.6f, y:%0.6f", msg.x, msg.y)


def pose_subscriber():

    # Initialize ROS node

    rospy.init_node('pose_subscriber', anonymous=True)

    # Create a subscriber, subscribe to the topic named /turtle1/pose, and register callback
function poseCallback rospy.Subscriber("/turtle1/pose", Pose, poseCallback)


    # Loop and wait callback function

    rospy.spin()


if __name__ == '__main__':

    pose_subscriber()
```

```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/scripts
File Edit View Terminal Tabs Help
1 #!/usr/bin/env python
2 # -*- coding: utf-8 -*-
3
4 import rospy
5 from turtlesim.msg import Pose
6
7 def poseCallback(msg):
8     rospy.loginfo("Turtle pose: x:%0.6f, y:%0.6f", msg.x, msg.y)
9
10 def pose_subscriber():
11     # ROS节点初始化
12     rospy.init_node('pose_subscriber', anonymous=True)
13
14     # 创建一个Subscriber, 订阅名为 /turtle1/pose的 topic, 注册回调函数 pose
    Callback
15     rospy.Subscriber("/turtle1/pose", Pose, poseCallback)
16
17     # 循环等待回调函数
18     rospy.spin()
19
20 if __name__ == '__main__':
21     pose_subscriber()W
: wq
```

- 3) Input “chmod +x pose_subscriber.py” command and press “Enter” to give the executable permission to saved pose_subscriber.py.

```
Terminal - ubuntu@ubuntu: ~/catkin_ws/src/beginner_hiwonder/scripts
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~/catkin_ws/src/beginner_hiwonder/scripts$ chmod +x pose_subscrib
r.py
```

2. Test Publisher and Subscriber

- 1) Input “roscore” command to start the node manager.

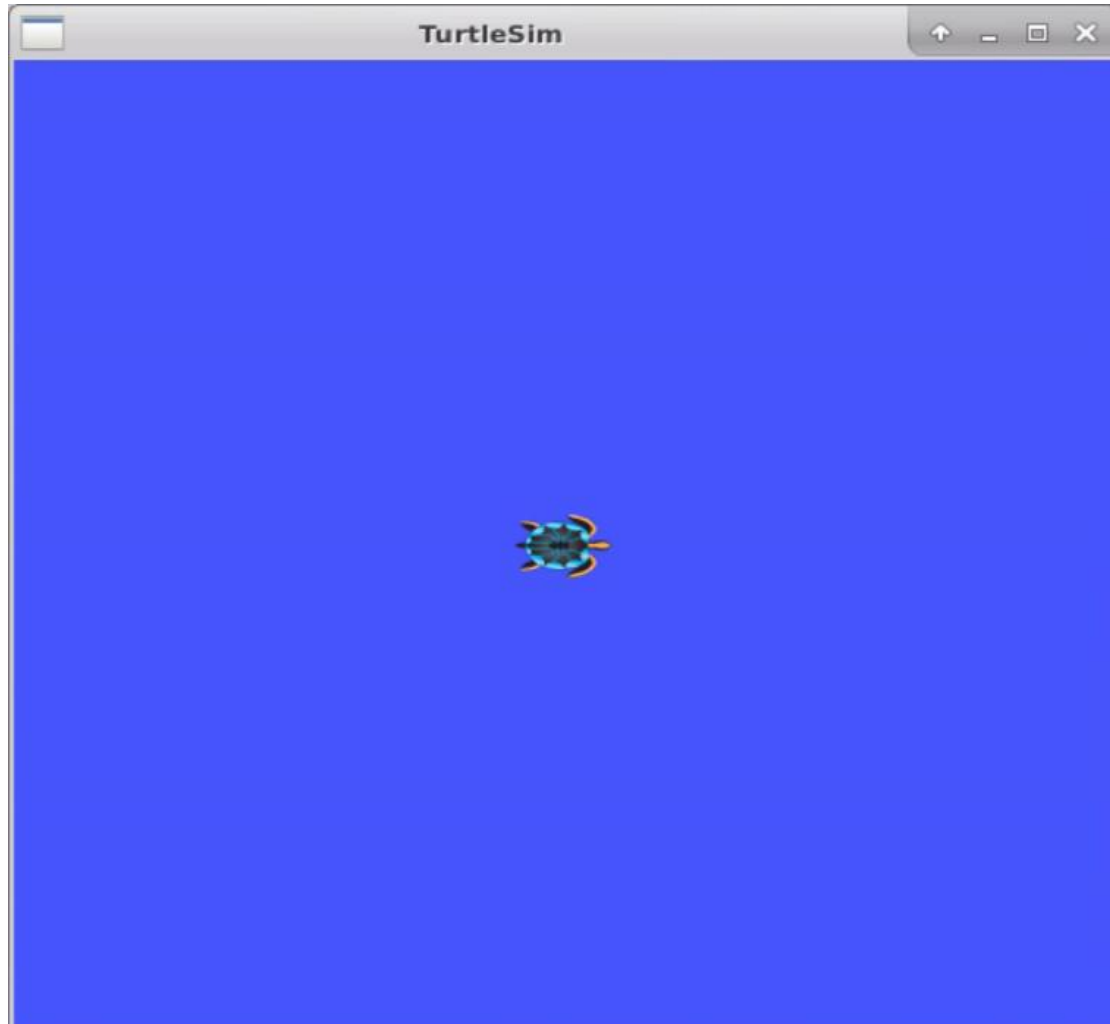
```
ubuntu@ubuntu:~/catkin_ws$ roscore
```

After starting, the following prompt will appear:

```
RLEException: roscore cannot run as another roscore/master is already running.
Please kill other roscore/master processes before relaunching.
The ROS_MASTER_URI is http://ubuntu:11311/
The traceback for the exception was written to the log file
ubuntu@ubuntu:~/catkin_ws$
```

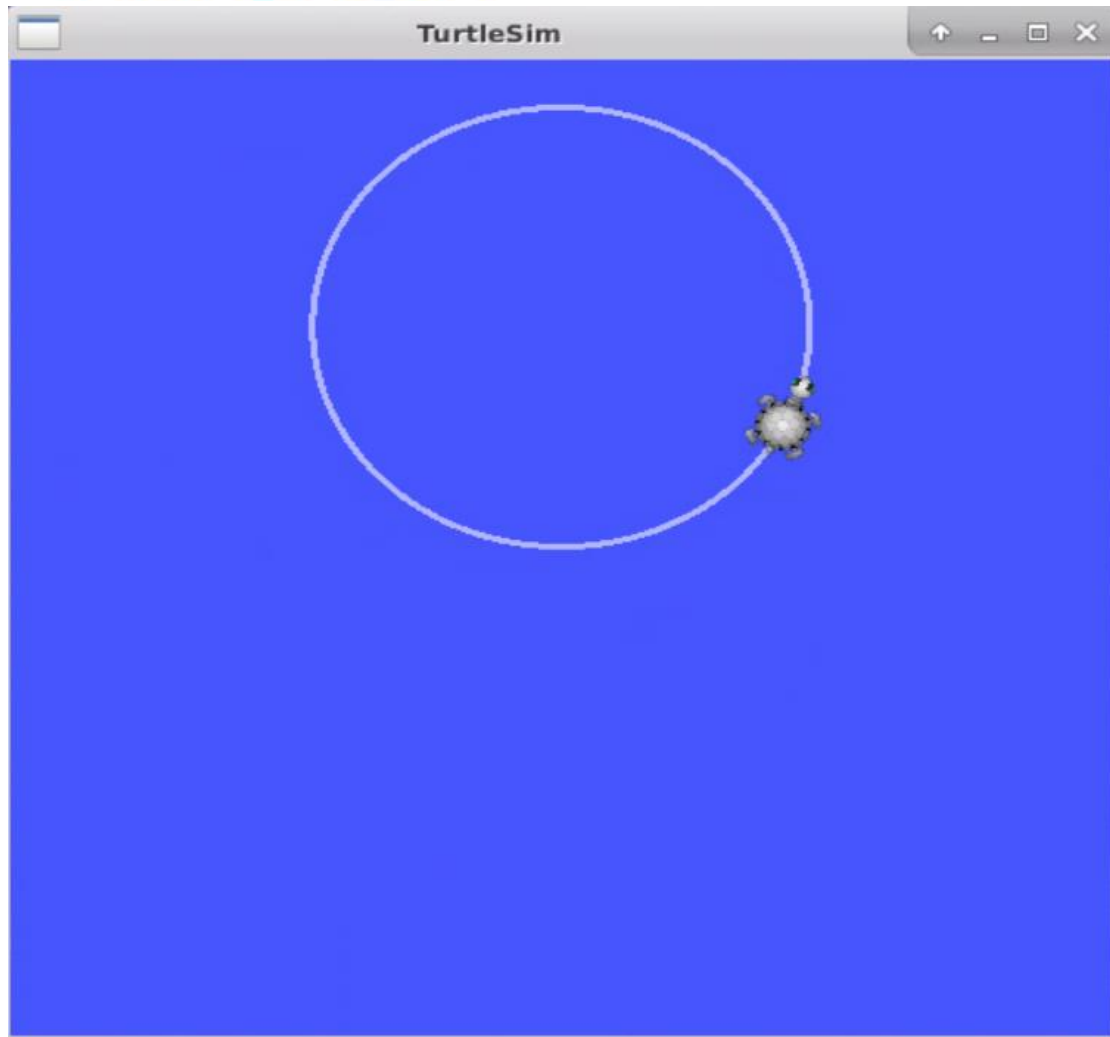
Input “roslaunch turtlesim turtlesim_node” command and then press “Enter” to start TurtleSim.

```
ubuntu@ubuntu:~/catkin_ws$ rosrn turtlesim turtlesim_node
libEGL warning: DRI2: failed to authenticate
[ INFO] [1644663888.439197118]: Starting turtlesim with node name /turtlesim
[ INFO] [1644663888.462207665]: Spawning turtle [turtle1] at x=[5.544445], y=[5.544445], theta=[0.000000]
```



Open a new terminal and enter “rosrn beginner_hiwonder velocity_publisher.py” command to run the publisher of velocity_publisher.py. Then press “Ctrl+C” to stop running the publisher node.

```
Terminal - ubuntu@ubuntu: ~
File Edit View Terminal Tabs Help
ubuntu@ubuntu:~$ rosrn beginner_hiwonder velocity_publisher.py
[INFO] [1644664960.154518]: Publshturtlevelocitycommand[0.50 m/s, 0.20 rad/s]
[INFO] [1644664960.255668]: Publshturtlevelocitycommand[0.50 m/s, 0.20 rad/s]
[INFO] [1644664960.355954]: Publshturtlevelocitycommand[0.50 m/s, 0.20 rad/s]
[INFO] [1644664960.460197]: Publshturtlevelocitycommand[0.50 m/s, 0.20 rad/s]
[INFO] [1644664960.556231]: Publshturtlevelocitycommand[0.50 m/s, 0.20 rad/s]
[INFO] [1644664960.656011]: Publshturtlevelocitycommand[0.50 m/s, 0.20 rad/s]
[INFO] [1644664960.757576]: Publshturtlevelocitycommand[0.50 m/s, 0.20 rad/s]
[INFO] [1644664960.863103]: Publshturtlevelocitycommand[0.50 m/s, 0.20 rad/s]
[INFO] [1644664960.959313]: Publshturtlevelocitycommand[0.50 m/s, 0.20 rad/s]
[INFO] [1644664961.055680]: Publshturtlevelocitycommand[0.50 m/s, 0.20 rad/s]
```



Open a new terminal and input “roslaunch beginner_hiwonder pose_subscriber.py” command to run the subscriber of pose_subscriber.py Then press “Ctrl+C” to stop running the subscribe node.

-
- ① The publisher node needs to be started first, and then the subscriber node can subscribe message.
 - ② If need to receive the publisher messages completely, you can start the subscriber node first and then the publisher node.
-